



復旦大學
FUDAN UNIVERSITY

算法设计与分析（第一讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

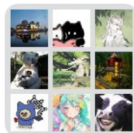
本人简介

- 计算与智能创新学院，青年研究员
- 研究方向：人工智能安全，数据隐私保护
- 办公室：复旦大学江湾校区二号交叉学科楼 D4027
- 个人主页：chenghuang-sec.github.io

助教

- 陈依琳, 25113050211@m.fudan.edu.cn
 - 朱钊毅, 26213050522@m.fudan.edu.cn
 - 罗瑞, 26113050220@m.fudan.edu.cn
 - 王意飞, 26213050373@m.fudan.edu.cn
-
- 助教答疑:
 - 微信/邮件: 早9点到晚9点, 工作日
 - 面对面交流: 周五(双)晚, 习题课 课间/结束后

课程微信群



群聊：算法26课程群



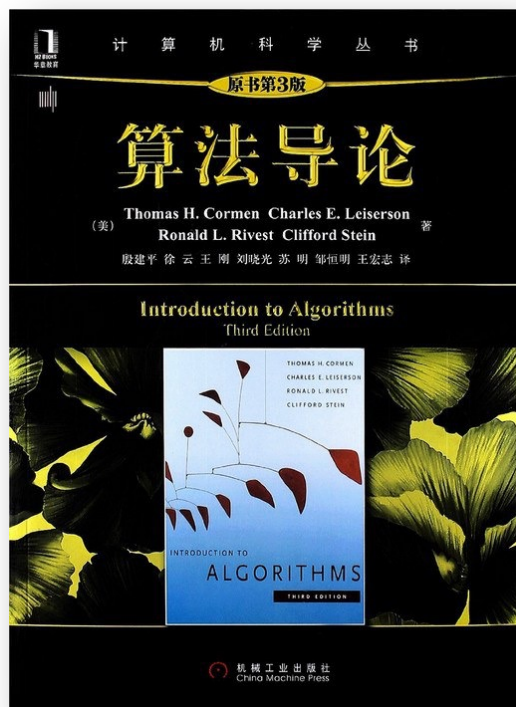
该二维码7天内(9月17日前)有效，重新进入将更新

考核

- 平时成绩（30%）
 - 考勤（10%）（不点名，有事请假）
 - 平时作业（20%，发布在eLearning上，通过eLearning系统上传，按时上传就有分）
- 期末考试（70%）
 - 不考察死记硬背，以考察概念掌握以及算法思维为主
 - 5道大题，每题20分

教材

- 《算法导论》(第3版)(美)科尔曼等著, 殷建平等译, 机械工业出版社



课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

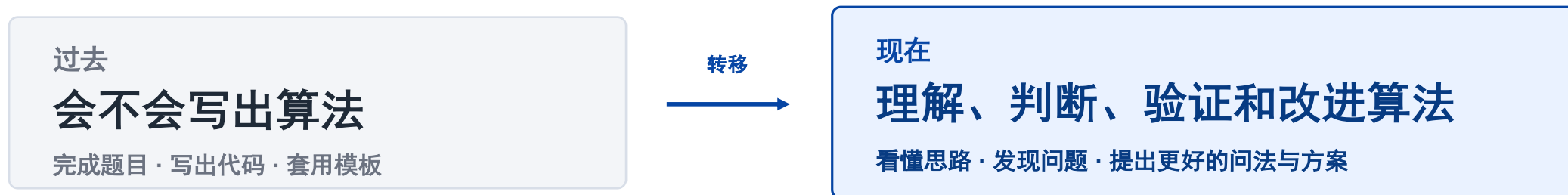
求解复杂算法问题

线性规划，在线算法，AI 算法

理解高效计算的边界

P vs NP, NP 完全性，近似算法

AI时代，算法课学什么？



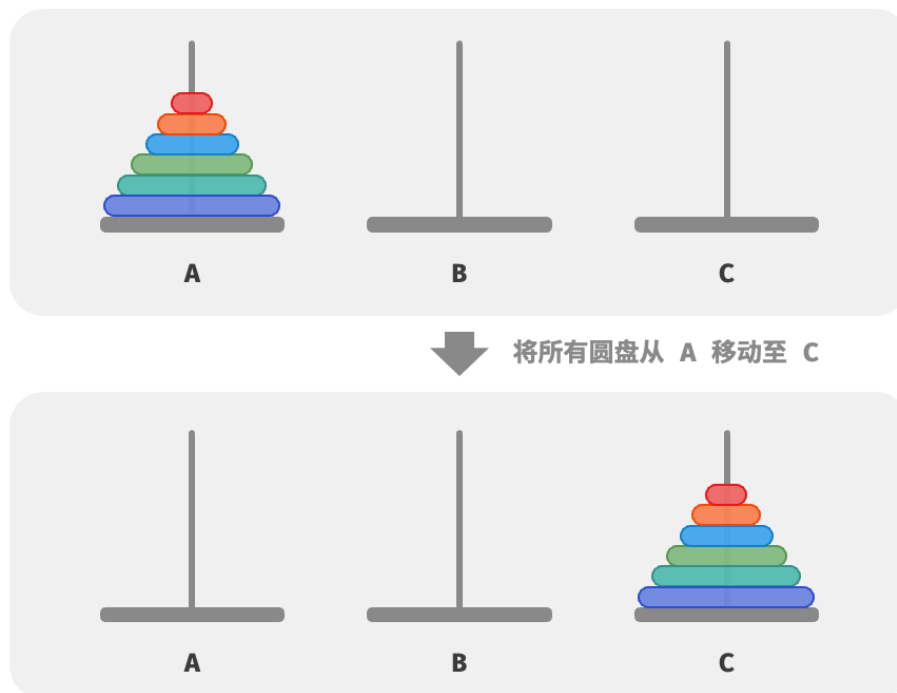
新的学习闭环：把 AI 变成“算法思考的放大器”



什么是算法？

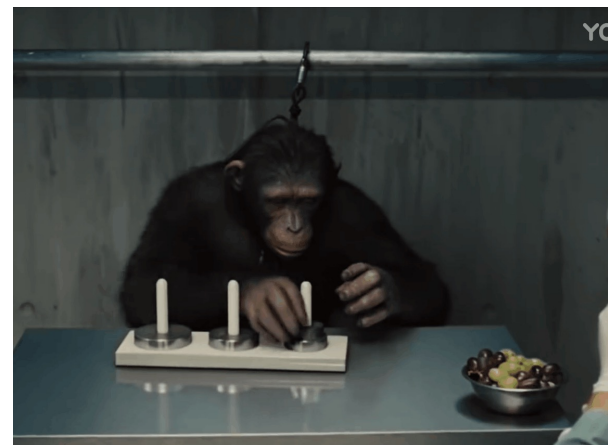


什么是算法？



汉诺塔游戏

n个圆盘要移动多少次？
比如n=64？



电影《猩球崛起》片段

什么是算法？

- 算法（Algorithm）是为解决某类计算问题而规定的一组明确、可执行的计算步骤。
- 通常具有以下特征：
 - 有穷性：对于给定的合法输入，应在有限步骤后终止。
 - 明确性：每一步操作及其执行规则都有明确、无歧义的定义。
 - 可执行性：每一步都是基本且可实际执行的计算操作。
 - 输入：接受零个或多个满足规定条件的输入。
 - 输出：产生满足问题要求、并与输入具有规定关系的结果。

问题

- 一次大模型推理，是算法吗？
 - 如果是算法，输入，计算过程，输出是什么？是确定性算法还是随机算法？如果不是算法原因是什么？
- 大模型推理会终止吗？
 - 如果会终止，终止条件是什么？

回答

- **1. 一次大模型推理，是算法吗？**
- 可以看作一次算法执行，但更准确地说：模型结构、参数、推理程序和解码策略共同构成推理算法。
- 输入：用户提示词、系统提示词、历史对话等上下文，以及模型参数和推理配置。
- 计算过程：
 - 将文本转换为 token；
 - 神经网络前向计算，得到下一个 token 的概率分布；
 - 使用贪心、采样或 Beam Search 选择 token；
 - 将新 token 加入上下文，重复上述过程。
- 输出：生成的 token 序列，最终表现为文本、代码、工具调用等。
 - 如果采用采样，同一输入可能产生不同输出，因此它属于随机算法
 - 如果采用贪心解码，可以近似看作确定性算法。

回答

■ 2. 大模型推理一定会终止吗？

■ 分两种情况：

- 补全回答/选择问题：生成一个 token：计算图规模有限，正常情况下会终止。

- 例如问：中国的首都是哪里？ A. 北京， B. 南京 请输出选择

- 连续生成一段文本：本质上是一个循环，不一定天然终止。

■ 实际系统通常设置以下终止条件：

- 模型生成结束标记 EOS；
- 达到最大生成长度 max_tokens；
- 遇到预设的停止字符串；
- 达到时间或计算资源限制；
- 用户或系统主动中止。

算法存在，不代表算法正确

- 对于所有合法输入，算法都应该得到满足问题要求的输出。
- 可以把算法正确性拆成两件事：
 - 1. 它最终会不会结束？
 - 2. 如果结束，它给出的答案是不是我们要的？

如果算法里有一个循环，可能执行 n 次、 10^6 次，我们怎么证明每一次都没有“跑偏”？

一个算法

算法目标：寻找数组里面的最大值

```
def find_max(A):  
    if len(A) == 0:  
        raise ValueError("列表不能为空")  
  
    m = 0  
  
    for x in A[0:]:  
        if x > m:  
            m = x  
  
    return m
```

正确性有问题吗？

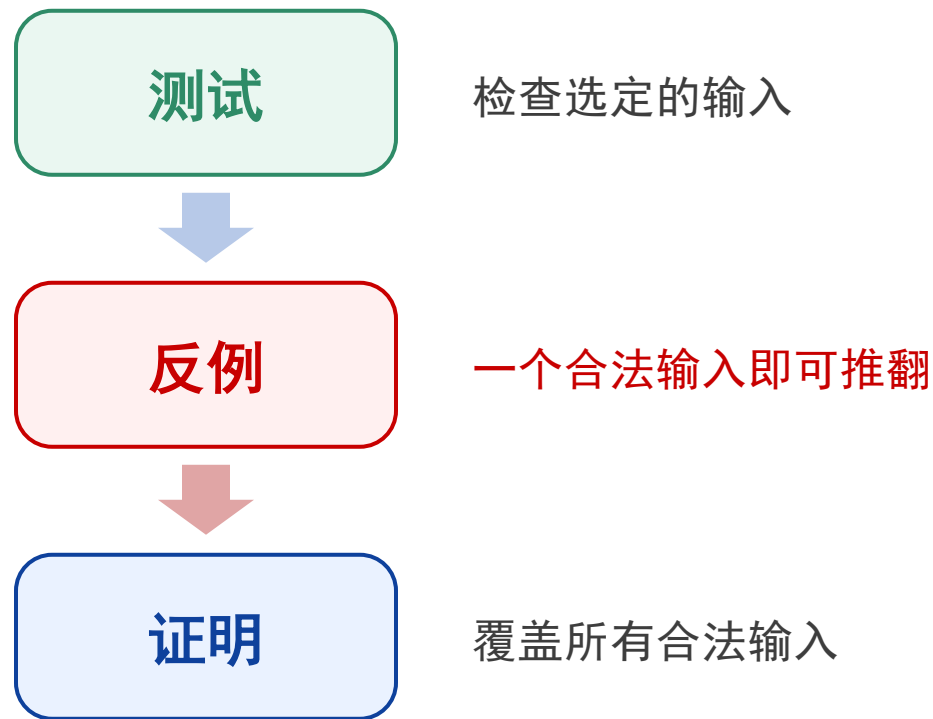
测试、反例与正确性分析

```
def find_max(A):  
    if len(A) == 0:  
        raise ValueError("列表不能为空")  
  
    m = 0  
  
    for x in A[0:]:  
        if x > m:  
            m = x  
  
    return m
```

测试: $A = [2, 7, 4]$ ✓

反例: $A = [-4, -2]$

算法输出 0, 正确答案 -2



$\forall A \in I$

有限测试帮助找错, 正确性证明需要覆盖所有合法输入

如何证明一个循环算法对所有输入都正确？

```
INSERTION-SORT(A)
```

```
for j ← 2 to length[A]  
    key ← A[j]  
    i ← j - 1
```

```
    while i > 0 and A[i] > key  
        A[i + 1] ← A[i]  
        i ← i - 1
```

```
    A[i + 1] ← key
```

这个“始终成立的性质”是什么？

需要找到一个在每一轮循环中始终保持成立的关键性质

循环不变式：证明循环始终没有“跑偏”

插入排序（简洁版）

核心思想：每次取一个元素，插入到左边已排序的部分中

已排序部分



开始：默认第 1 个元素本身已经有序。

绿色：已经有序 黄色：当前准备插入的元素

在第 j 轮开始前， $A[1..j-1]$ 已经有序，并且包含原数组前 $j-1$ 个元素。

这个性质就是 **“循环不变式”**

循环不变式主要用来干什么？

- 证明循环算法的正确性（本质上是把数学归纳法应用到了程序循环上）
- 归纳基础：循环开始前，不变式成立；
- 归纳步骤：如果某一轮开始前成立，执行这一轮后仍然成立；
- 利用最终状态得出结论：循环结束时，利用不变式推出最终结果正确。

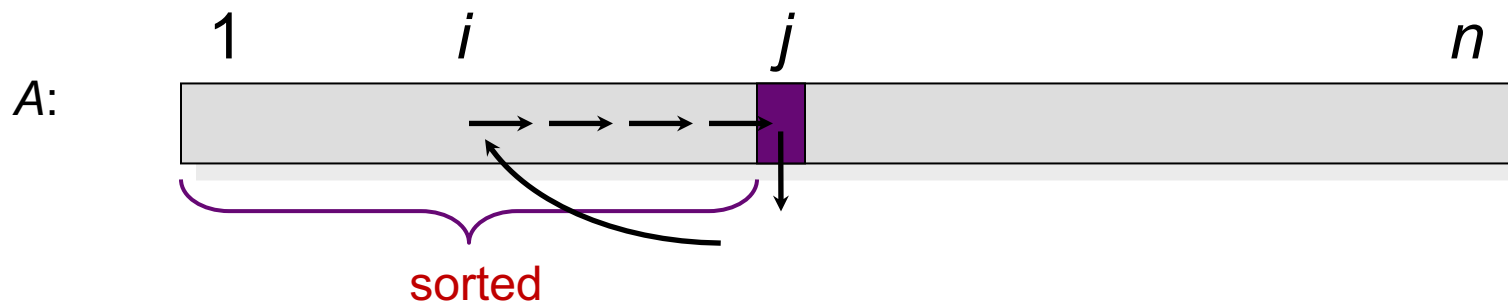
算法设计与分析

- 算法设计：针对计算问题，设计正确、可行的求解方法
- 算法分析：评估算法所需的时间、空间等计算资源

**在保证正确性的前提下，分析算法开销，
并通过更优的设计降低复杂度、提升效率。**

插入排序

```
def insertion_sort(A):  
    for j in range(1, len(A)):  
        key = A[j]  
        i = j - 1  
  
        while i >= 0 and A[i] > key:  
            A[i + 1] = A[i]  
            i -= 1  
  
        A[i + 1] = key
```



运行时间是否能衡量算法的性能？

- ① 输入会影响运行时间吗？
 - 输入内容 + 输入规模 n
- ② 为什么不能只看“运行了几秒”？
 - 机器、语言、编译器、实现都会影响实际时间
- ③ 如何比较算法本身？
 - 统计基本操作次数，并分析其随 n 的增长规律

我们关心的不是“这次跑了几秒”，而是“当问题规模变大时，计算量增长得有多快”。

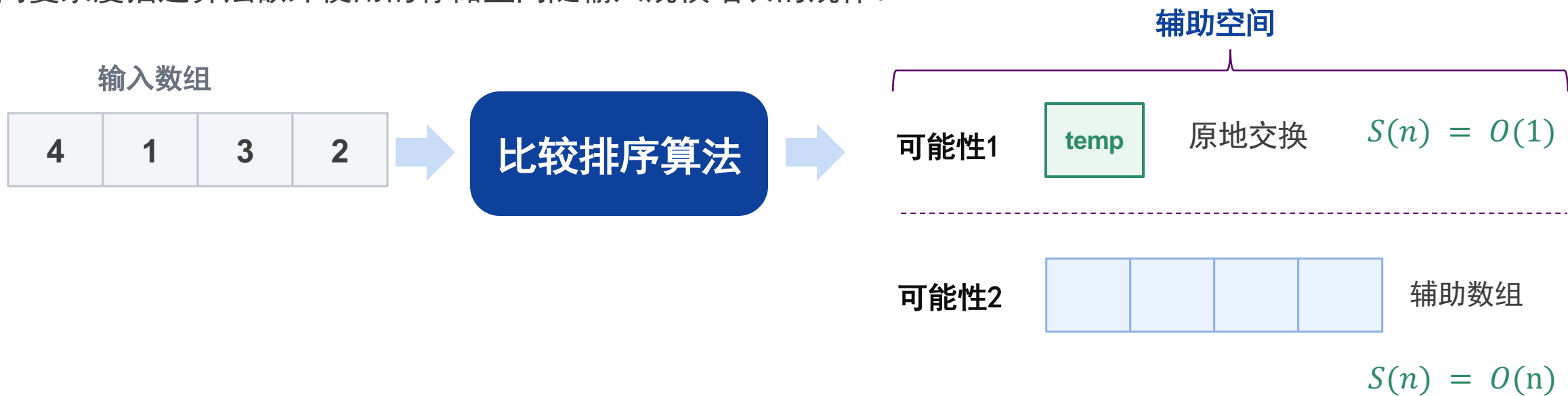
时间复杂度

- 时间复杂度描述算法所需基本操作次数随输入规模 n 增长的规律。
- 回答三个问题
 - 1. 输入规模是什么？ 如：数组长度 n
 - 2. 基本操作是什么？ 如：比较、移动
 - 3. 随着 n 增大，操作次数如何增长？ $T(n) = ?$

强调一下：时间复杂度关注“增长规律”，而不是某一次运行用了多少秒。

空间复杂度

空间复杂度描述算法额外使用的存储空间随输入规模增长的规律。



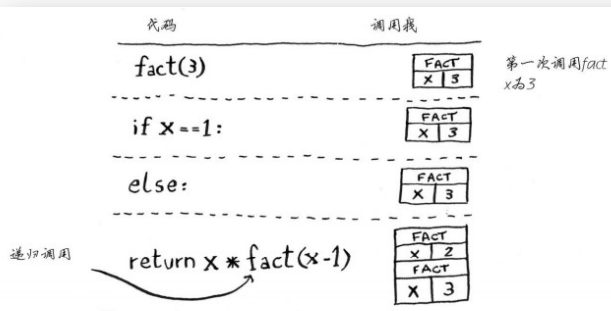
通常分析辅助空间，不重复计算输入本身。

速度相同的算法，也可能需要完全不同的内存

空间复杂度

```
def fact(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fact(x-1)
```

递归调用也要占用额外空间：
递归调用栈



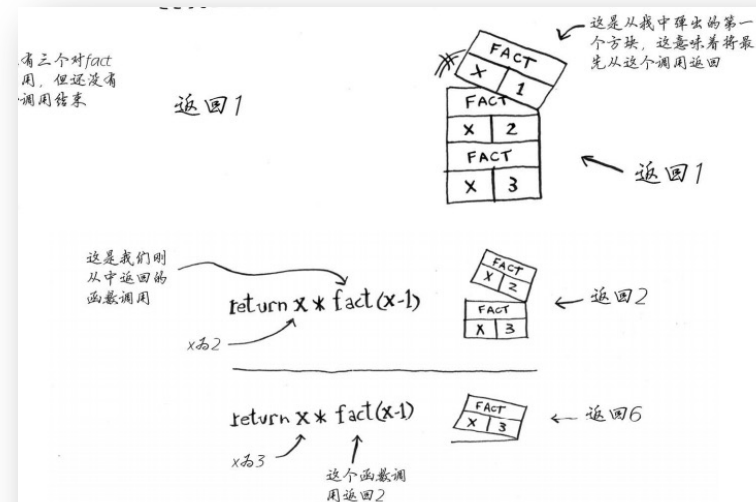
现在位于对fact的第二
次调用中。x为2

if x==1:

else:

return x * fact(x-1)

if x==1:



时间复杂度

- 最坏情况下的时间复杂度（Worst-Case）
 - 算法求解输入规模为 n 的实例所需要的**最长时间** $T(n)$
- 平均情况下的时间复杂度（Average-Case）
 - **在指定输入的概率分布下**，算法求解输入规模为 n 的实例所需要的**平均时间** $T(n)$
- 最好情况下的时间复杂度（Best-Case）
 - 算法求解输入规模为 n 的某些特定实例所需要的**最快时间** $T(n)$

情况	定义	插入排序
Worst Case	$\max T(I)$	完全逆序
Average Case	$\mathbb{E}[T(I)]$	给定输入分布下的期望
Best Case	$\min T(I)$	已经有序

插入排序--最坏情况下的时间复杂度

- **最坏情况**发生在输入序列是**逆序**时。每次插入一个元素时，都需要将之前的所有元素向后移动。
 - 第1次插入时，比较1次。
 - 第2次插入时，比较2次。
 - ...
 - 第n-1次插入时，比较n-1次。
- 总比较次数是一个等差数列的和
 - $T(n) = 1 + 2 + 3 + \dots + (n - 1) = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$

我们每分析一个算法，都要保留 $\frac{1}{2}n^2 - \frac{1}{2}n$ 这么精确吗？

从精确复杂度到增长趋势

插入排序最坏情况: $T(n) = \frac{1}{2}n^2 - \frac{1}{2}n$  当 n 足够大时: n^2

忽略与机器和实现相关的常数，以及低阶项，关注算法运行成本随输入规模增长的趋势。

渐近分析 / 渐近符号 – **Big- Θ**

- 渐近分析关心的是：当输入规模足够大时，运行成本如何增长。
- 渐近符号 Θ ，定义为“渐近紧确界”
- $f(n) = \Theta(g(n))$ 意味着 $f(n)$ 和 $g(n)$ 在“增长趋势”上是同一个量级

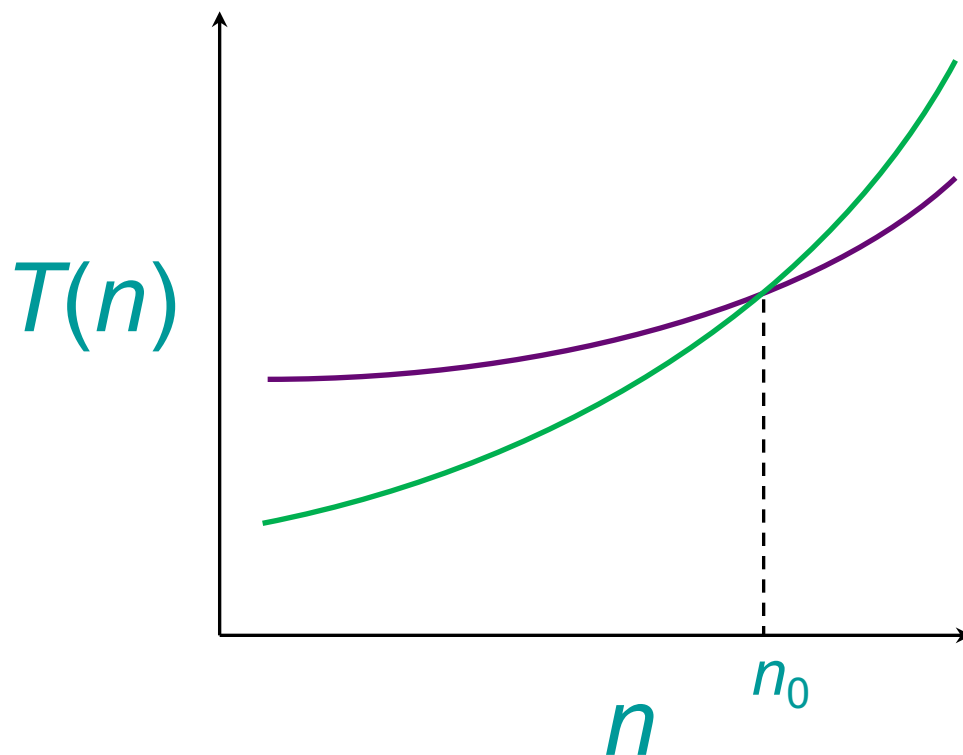
设 f 和 g 是定义域为自然数集 N 上的函数，若存在正数 c_1, c_2 和 n_0 ，使得对一切 $n \geq n_0$ 有 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ 成立，则称 $f(n)$ 的一个渐近紧确界是 $g(n)$ ，记作 $f(n) = \Theta(g(n))$

插入排序算法的一个渐近紧确界

- 插入排序的复杂度: $\frac{1}{2}n^2 - \frac{1}{2}n$
- 需要找到 c_1, c_2, n_0 , 满足: $c_1n^2 \leq T(n) = \frac{1}{2}n^2 - \frac{1}{2}n \leq c_2n^2$
- 因为 $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$, 所以可以取: $\boxed{c_2 = \frac{1}{2}}$
- 当 $n \geq 2$ 时: $n - 1 \geq \frac{n}{2}$, 所以: $\frac{1}{2}n^2 - \frac{1}{2}n = \frac{n(n-1)}{2} \geq \frac{n(n/2)}{2} = \frac{1}{4}n^2$, 因此可以取: $\boxed{c_1 = \frac{1}{4}}$
- 同时: $\boxed{n_0 = 2}$
- 所以根据 Θ 的定义: $\boxed{T(n) = \Theta(n^2)}$
- c_1, c_2, n_0 不是唯一的, 我们只需要证明存在一组合适的常数即可。

渐近符号为什么足够描述性能?

- 当 n 足够大时, $\Theta(n^2)$ 算法总是比 $\Theta(n^3)$ 算法性能要好



$$100n$$

$$n^2 + 3n$$

$$2n^2 + 3n$$

$$n^3$$

等式中的渐近符号

- $2n^2 + 3n + 1 = 2n^2 + f(n)$ 其中 $f(n) = 3n + 1 \in \Theta(n)$
- 因此可以写成：

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

- 这里的 $\Theta(n)$ 可以理解为：一个具体形式被隐藏的、属于 $\Theta(n)$ 的匿名函数
- 由于低阶项不改变整体增长阶：

$$2n^2 + \Theta(n) = \Theta(n^2)$$

- 渐近符号允许我们隐藏不影响增长阶的细节，只保留对复杂度分析真正重要的信息。

归并排序的渐近分析

MERGE-SORT $A[1 \dots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \dots \lceil n/2 \rceil]$ and $A[\lceil n/2 \rceil + 1 \dots n]$.
3. “**Merge**” the 2 sorted lists.

关键子步骤: “**Merge**”

归并排序的渐近分析

归并排序 (Merge Sort)

核心思想：先分解，再两两合并

原始数组



开始：把待排序数组不断对半拆分，直到每个子数组只剩 1 个元素。

 当前正在拆分或合并的部分

 已经合并完成的部分

合并2个排序好的序列的渐近分析

归并排序：合并两个有序分片

核心思想：不断比较两个分片的“当前最小值”

左有序分片



右有序分片



合并结果



现在只看归并排序中的“合并”步骤：把两个已经排好序的分片合成一个更大的有序分片。

如果是合并两个已经排好序的分片，假设它们长度分别是 m 和 n ，那么：

$$\Theta(m + n)$$

因为合并时两个指针都只会从左向右走一遍：

- 左分片每个元素最多访问一次；
- 右分片每个元素最多访问一次；
- 每次比较后，至少有一个指针向前移动。

所以总操作数与两个分片元素总数成正比。

如果两个分片各自大约是 $n/2$ ，那么就是： $\Theta(n)$

归并排序的渐近分析

$$\begin{array}{l} T(n) \\ \Theta(1) \\ 2T(n/2) \\ \nearrow \\ \Theta(n) \end{array}$$

MERGE-SORT $A[1 \dots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \dots \lceil n/2 \rceil]$ and $A[\lceil n/2 \rceil + 1 \dots n]$.
3. **“Merge”** the 2 sorted lists

应该是 $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$

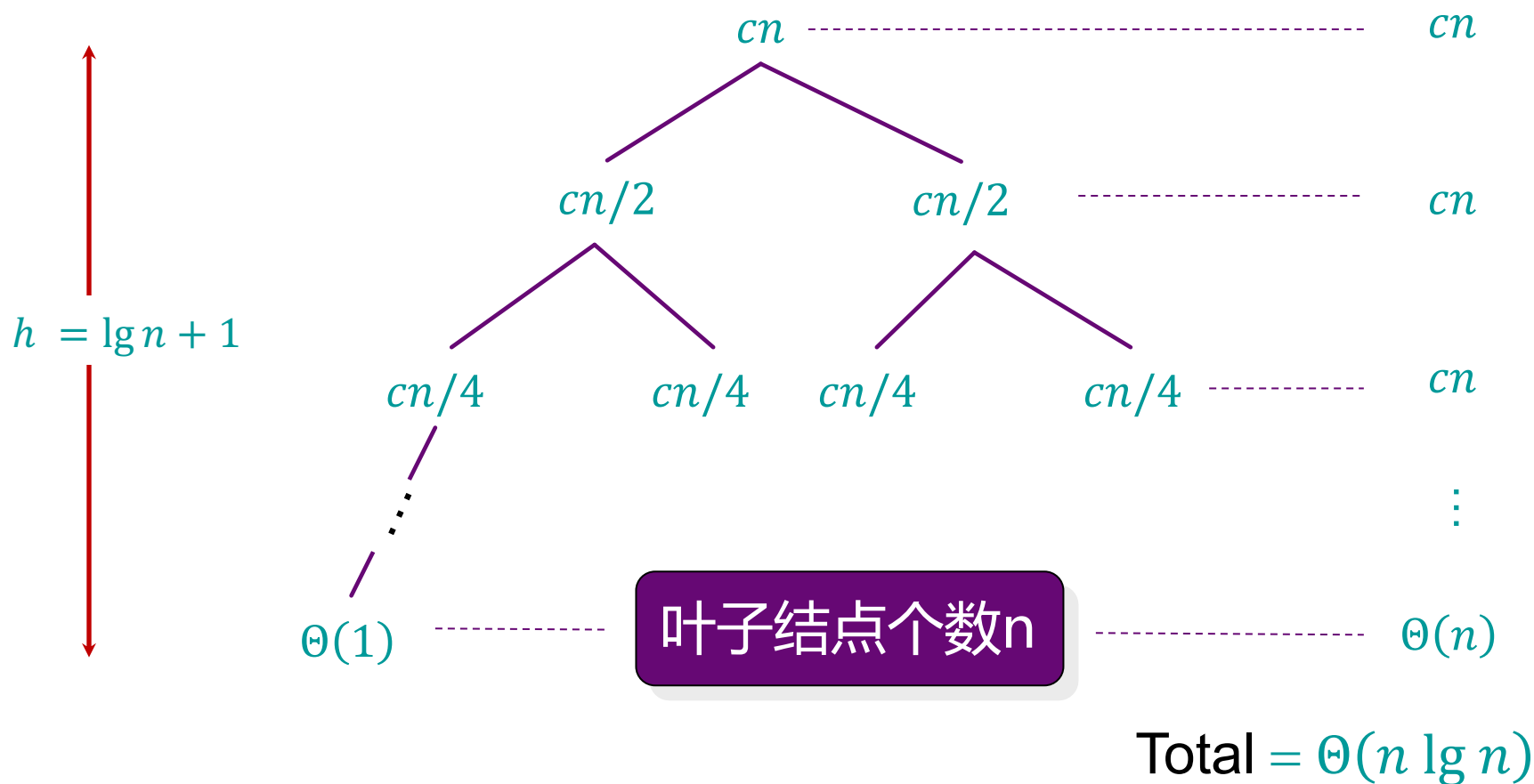
归并排序的递归式（递归方程）

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & n > 1 \end{cases}$$

如何求解？ 递归树

递归树求解

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \quad \text{这里 } c > 0, \text{ 且是个常数}$$



归并排序的渐近性能

- $\Theta(n \log n)$ 比 $\Theta(n^2)$ 增长慢。
- 因此，最坏情况下归并排序的渐近性能比插入排序好。

递推式的三部分，正好对应树的三种信息

$$T(n) = a \cdot T(n/b) + f(n)$$

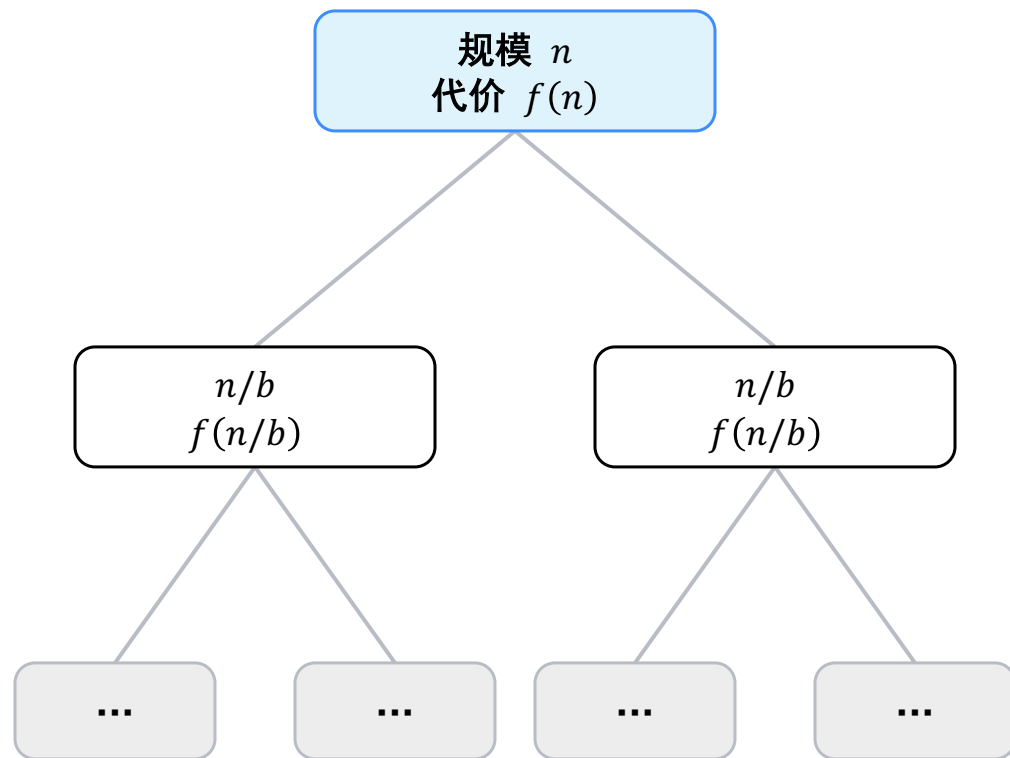
- **a : 分支数**

每个节点产生 a 个子问题。

- **b : 缩小倍数**

子问题规模从 n 变成 n/b 。

- **$f(n)$: 节点代价**



树的总成本 = 所有节点代价之和

解递归只需回答三个问题

STEP 1

展开：每个节点是什么？

写出第 i 层的节点数、子问题规模与单节点代价。

STEP 2

按层：这一层一共多贵？

计算本层成本：节点数 $\times$ 单节点代价。先看它随 i 如何变化。

STEP 3

收尾：树高与叶子有多少？

由 $n/b^h \approx 1$ 得 $h \approx \log_b(n)$ 。把内部层与叶子层一起求和。

先算“每层”，再数“多少层”；递归树就从图形变成求和式。

渐近上界 – Big-O

若存在正数 c 和 n_0 ，使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立，则称 $f(n)$ 的一个渐近上界是 $g(n)$ ，记作 $f(n) = O(g(n))$

例： $2n^2 = O(n^3)$ ($c = 1, n_0 = 2$)

例： $2n^2 = O(n^2)$ ($c = 3, n_0 = 1$)

例： $2n^2 = O(n^4)$ ($c = 1, n_0 = 2$)

更“松”的上界

更“紧”的上界

也可以写成： $2n^2 \in O(n^3)$

可以”看成“一个函数集合 $O(g(n)) = \{f(n) : \exists c > 0, n_0, 0 \leq f(n) \leq cg(n), \forall n \geq n_0\}$

渐近下界 – **Big-Ω**

若存在正数 c 和 n_0 ，使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立，则称 $f(n)$ 的一个渐近下界是 $g(n)$ ，记作 $f(n) = \Omega(g(n))$

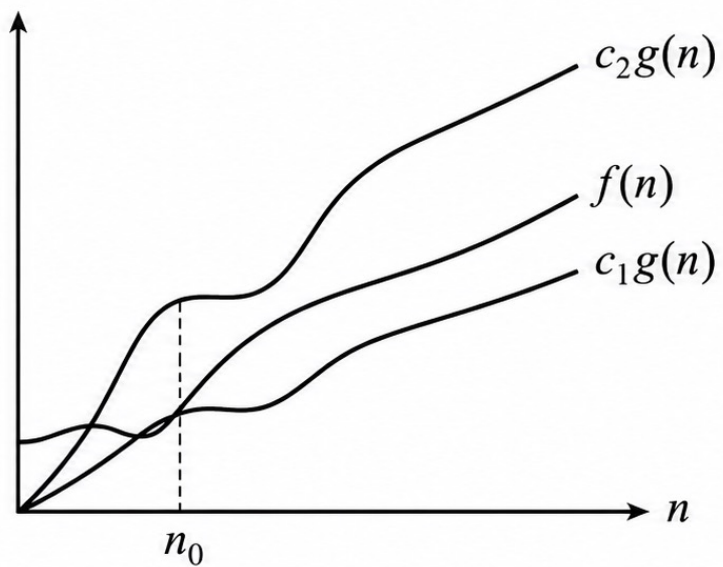
$$\text{例： } \sqrt{n} = \Omega(\lg n) \quad (c = 1, n_0 = 16)$$

$$\text{例： } \sqrt{n} = \Omega(\sqrt[3]{n}) \quad (c = 1, n_0 = 1)$$

$$f(n) = O(g(n)) \text{ 和 } f(n) = \Omega(g(n)) \xrightarrow{\text{等同于}} f(n) = \Theta(g(n))$$

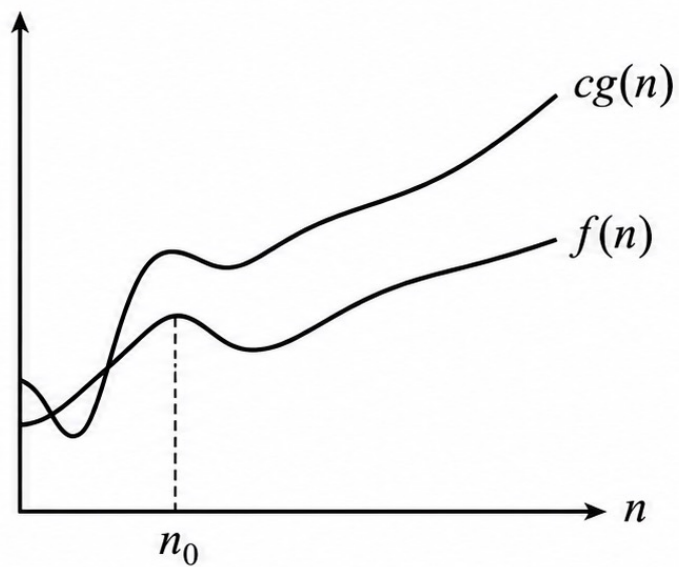
$$\Rightarrow \Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$

渐近符号的关联性



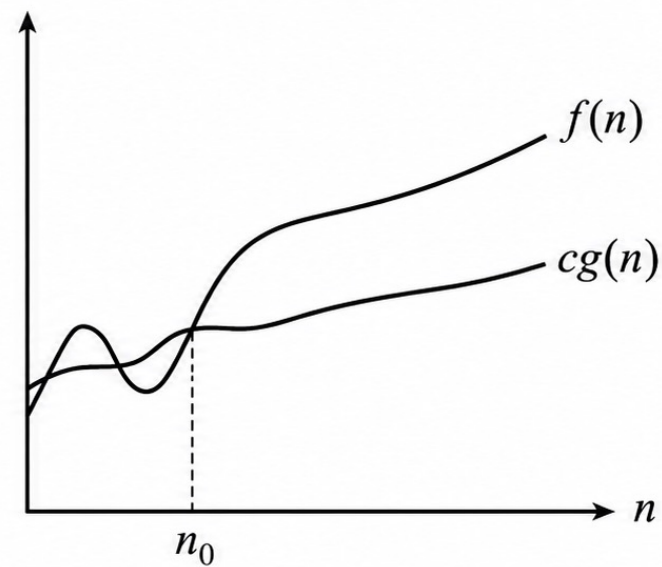
$$f(n) = \Theta(g(n))$$

(a)



$$f(n) = O(g(n))$$

(b)



$$f(n) = \Omega(g(n))$$

(c)

非渐近紧确上界/下界 – **Little-o/ ω**

非渐近紧确上界：对任意正数 c 都存在 $n_0 > 0$ ，使得当 $n \geq n_0$ 有 $0 \leq f(n) < cg(n)$ 成立，则记作 $f(n) = o(g(n))$

$$\text{例： } 2n^2 = o(n^3) \quad (n_0 = 3/c)$$

非渐近紧确下界：对任意正数 c 都存在 $n_0 > 0$ ，使得当 $n \geq n_0$ 有 $0 \leq cg(n) < f(n)$ 成立，则记作 $f(n) = \omega(g(n))$

$$\text{例： } n = \omega(\sqrt{n}) \quad (n_0 = c^2)$$

Big-O 并不等于 “最坏情况”

- 归并排序无论输入本来是有序、逆序，还是随机，都会先不断二分，再逐层合并。因此：

$$T_{\text{best}}(n) = \Theta(n \log n)$$

$$T_{\text{avg}}(n) = \Theta(n \log n)$$

$$T_{\text{worst}}(n) = \Theta(n \log n)$$

- 也就是说，**Big-O** 不是 “最坏情况” 的同义词。这里的 O 描述的是 “最好情况函数” 的渐进上界，并不是在说最坏情况。
- $T_{\text{best}}(n)$ 、 $T_{\text{avg}}(n)$ 、 $T_{\text{worst}}(n)$ 这三个函数都可以分别用 O, Θ, Ω 描述。
- 所以严格来说，应该分两步：

先选情况：best / average / worst

，

再用 O, Ω, Θ 描述增长率

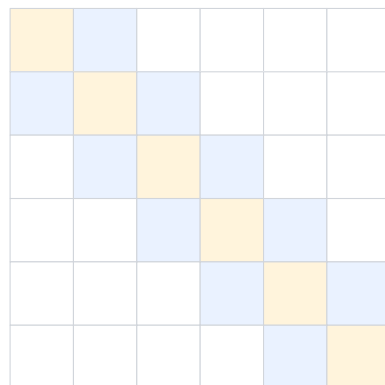
AI 时代，渐近符号用在哪？

复杂度分析不是为了背公式，而是为了判断：

规模变大时，计算量会怎样增长？



Token 变多会怎样?



$n \times n$ attention matrix

$$Q, K \in R^{n \times d}$$

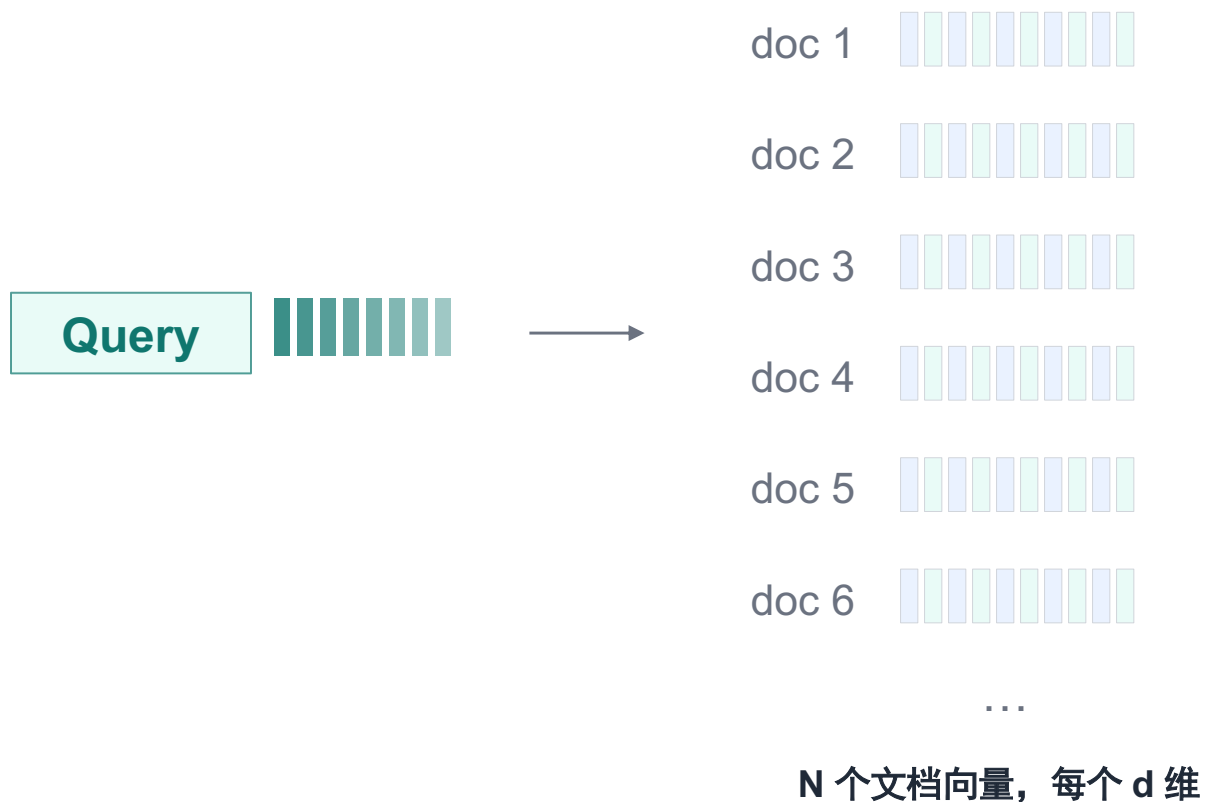
$$QK^T \rightarrow O(n^2d)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Attention: 每个 token 都要和其他 token 建立关系

关键增长: n 从 1K 到 10K, n^2 这一项约放大 100 倍

数据变多会怎样？



朴素检索：逐个比较相似度

一次相似度： $O(d)$

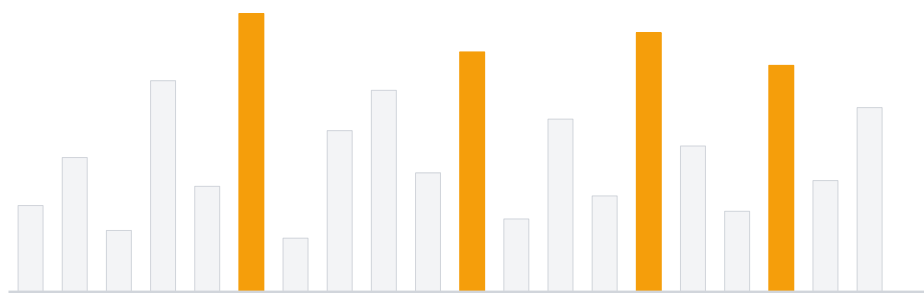
N 个文档： $O(Nd)$

当 N 很大时，不能总想着“全扫一遍”。

复杂度视角：N 控制“要看多少数据”，d 控制“每次比较多贵”。

候选很多，但只要少数怎么办？

例：N = 50,000 个候选 token 分数



只取最高的 k 个

方案 A：全部排序

$$O(N \log N)$$

排出了完整顺序，但很多信息用不上

方案 B：维护大小为 k 的堆

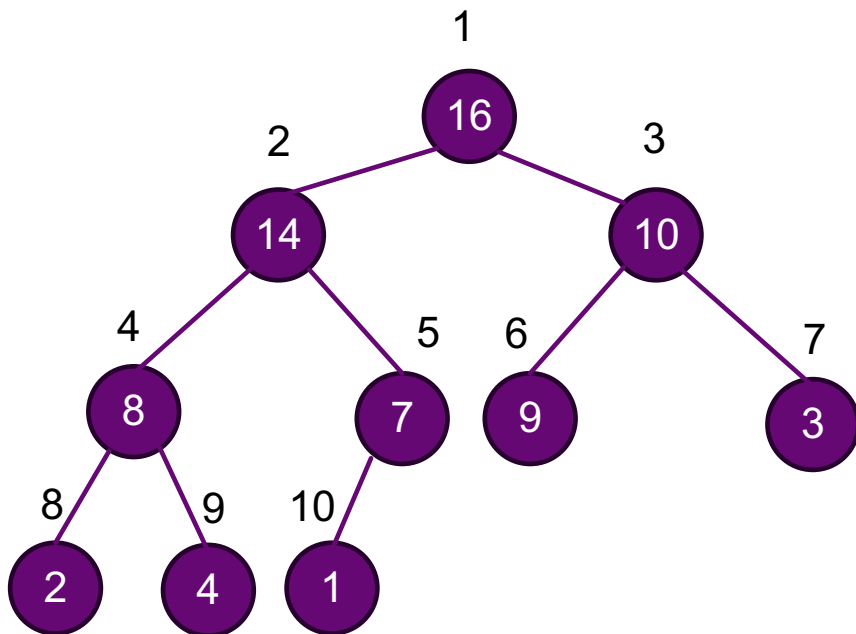
$$O(N \log k)$$

只关心前 k 个，避免完整排序

算法设计问题：目标变了，最优方法也会变。

堆

- 一个完全二叉树（数组）
- 最大堆性质：一个节点的键值大于或等于其子节点的键值（最小堆的定义类似）。



$parent(i) = i / 2$: 返回节点的父节点的索引

$left(i) = 2 * i$: 返回节点的左子节点的索引

$right(i) = 2 * i + 1$: 返回节点的右子节点的索引

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

堆的基本操作

- BUILD-HEAP: 从无序的数组中构造（最大）堆
- MAX-HEAPIFY: 维护最大堆性质，纠正子树根节点处堆性质的一处违规情况
- HEAPSORT: 对一个数组进行原址排序

MAX-HEAPIFY 操作

```
MAX-HEAPIFY(A, i):  
    l = left(i)  
    r = right(i)  
  
    if l <= heap-size(A) and A[l] > A[i]:  
        largest = l  
    else:  
        largest = i  
  
    if r <= heap-size(A) and A[r] > A[largest]:  
        largest = r  
  
    if largest != i:  
        exchange A[i] and A[largest]  
        MAX-HEAPIFY(A, largest)
```

一句话：把一个过小的节点一路“下沉”，直到最大堆性质恢复。

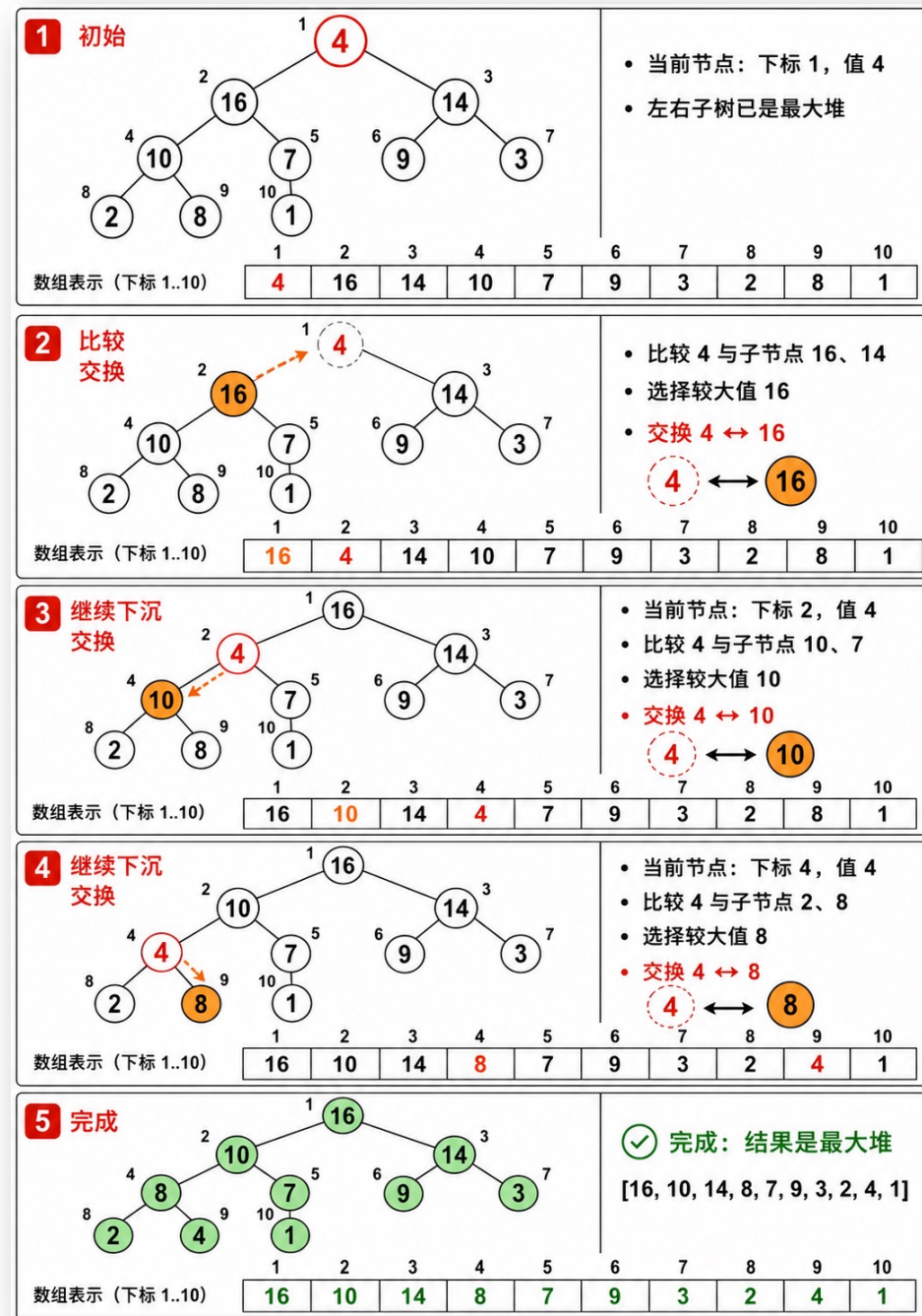
- 前提：节点 i 的左、右子树都已经是最大堆，只有节点 i 本身可能放错了位置。
- 做法：比较 $A[i]$ 和它的两个孩子。如果孩子中有更大的元素，就把最大的孩子和 $A[i]$ 交换；然后继续向下检查，直到 $A[i]$ 到达合适的位置。

MAX-HEAPIFY例子



左右子树没问题，但 3 比孩子小，所以把它和最大的孩子 10 换

- MAX-HEAPIFY 的复杂度：每次最多向下一层，而最大堆高度是 $\Theta(\log n)$ 。
- 所以最坏时间复杂度是： $\Theta(\log n)$

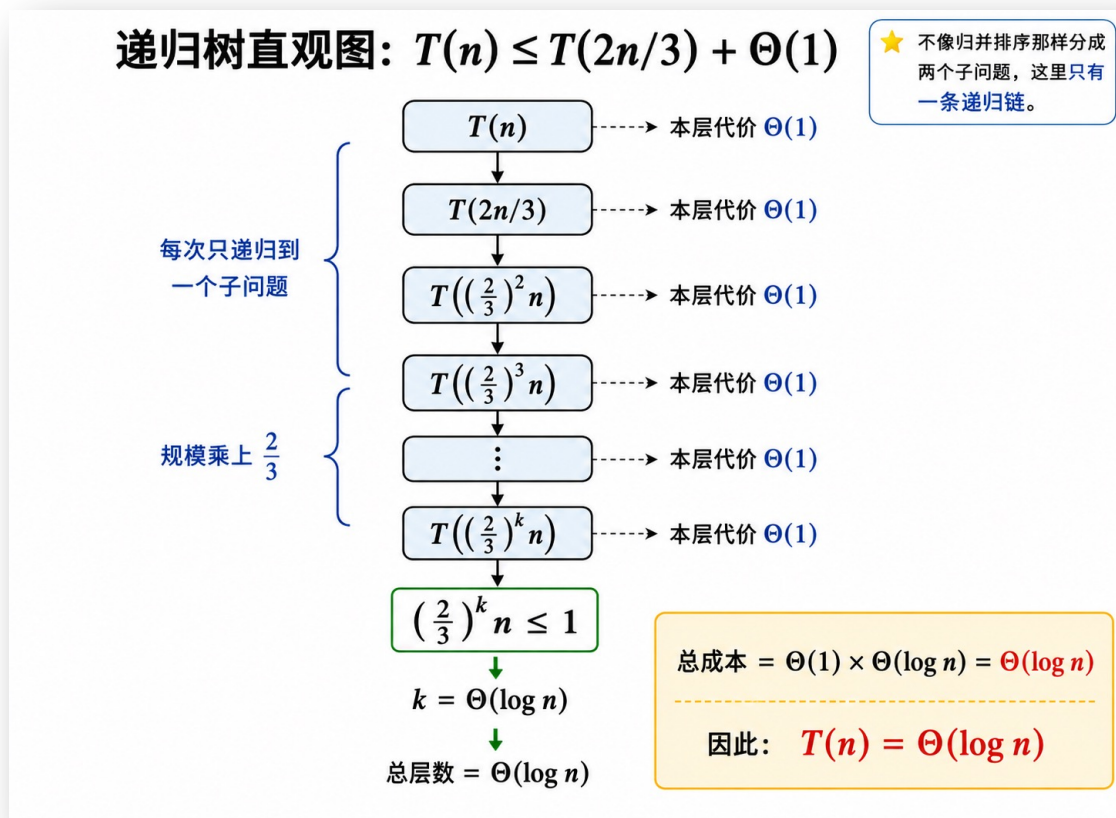


MAX-HEAPIFY的复杂度分析

- 因为堆是**完全二叉树**，左右子树虽然不一定一样大，但大小差距受到了很强的限制
- 在最不平衡的情况下，较大的那个子树也不会超过总节点数的 $2/3$
- **分析：**假设当前堆的高度是 h 。为了让左子树尽可能大、右子树尽可能小，最极端的情况是：最后一层的节点全部尽量塞到左子树。
 - 设左子树达到满的状态，那么： $|left| = 2^h - 1$ ；右子树至少包含前 $h - 1$ 层，因此： $|right| = 2^{h-1} - 1$
 - 再加根节点，总节点数至少是： $n = 1 + (2^h - 1) + (2^{h-1} - 1)$ ，即： $n = 3 \cdot 2^{h-1} - 1$
 - 而最大的左子树： $n_L = 2^h - 1$ ，得到 $\frac{n_L}{n} = \frac{2^h - 1}{3 \cdot 2^{h-1} - 1} < \frac{2}{3}$ ，所以： $\boxed{n_L \leq \frac{2n}{3}}$

MAX-HEAPIFY的复杂度分析 (续)

- 最坏情况下，较大的那个子树规模不会超过大约： $\frac{2n}{3}$ ，所以： $T(n) \leq T(2n/3) + \Theta(1)$



建最大堆 **BUILD-HEAP**

- 把A[1,...,n]转换成最大堆

```
BUILD-HEAP (A):  
  for i =  $\lfloor A.length/2 \rfloor$  downto 1  
    MAX-HEAPIFY(A, i)
```

- 复杂度是 $O(n \log n)$ 吗?

包含n个元素的堆的高度

- 高度为 H 的完全二叉树，最多的节点数是： $\sum_{i=0}^H 2^i = 2^{H+1} - 1$
- 高度为 H 的完全二叉树，最少的节点数是： $\sum_{i=0}^{H-1} 2^i + 1 = 2^H - 1 + 1 = 2^H$
- $2^H \leq n \leq 2^{H+1} - 1$
- 左边： $2^H \leq n \Rightarrow H \leq \lg n$
- 右边： $n \leq 2^{H+1} - 1 \Rightarrow H \geq \lg(n + 1) - 1$
- $\lg(n + 1) - 1 \leq H \leq \lg n \Rightarrow H = \lfloor \lg n \rfloor$
- 含 n 个元素的堆的高度为 $\lfloor \lg n \rfloor$

高度为 h 的结点有多少?

- 设堆用 $A[1 \dots n]$ 存储, 对于结点 i : $\text{left}(i) = 2i, \text{right}(i) = 2i + 1$
- 如果结点 i 的高度至少为 h , 说明从它往下至少还能走 h 层
- 沿着左孩子一直走 h 次, 最左侧后代的下标是: $i \cdot 2^h$
- 这个结点必须存在, 因此: $i \cdot 2^h \leq n$, 也就是: $i \leq \frac{n}{2^h}$
- 所以, 高度至少为 h 的结点数是: $\left\lfloor \frac{n}{2^h} \right\rfloor$
- 高度至少为 h 的结点里, 还要减去高度至少为 $h + 1$ 的结点: $N_h = \left\lfloor \frac{n}{2^h} \right\rfloor - \left\lfloor \frac{n}{2^{h+1}} \right\rfloor$
- 而这个值至多为: $\left\lfloor \frac{n}{2^{h+1}} \right\rfloor$, 所以: $N_h \leq \left\lfloor \frac{n}{2^{h+1}} \right\rfloor$

为什么“高度至少为 h 的结点里, 要减去高度至少为 $h+1$ 的结点”?

示意图 (完全二叉树)

高度 ≥ 3 (只有结点 1)

高度 ≥ 2 (结点 1, 2, 3)

高度 ≥ 1 (结点 1-7)

高度 ≥ 0 (结点 1-15)

这是一个高度为 3 的完全二叉树 (共 15 个结点)

1. 什么是“高度至少为 h 的结点”?

高度至少为 h 的结点: 从该结点出发, 至少存在一条向下的路径, 包含 h 条边。

2. “高度恰好为 h ” 怎么计算?

高度恰好为 h 的结点 = (高度至少为 h 的结点) - (高度至少为 $h+1$ 的结点)

3. 用图中的例子说明

- 高度 ≥ 3 : {1} (1 个)
- 高度 ≥ 2 : {1, 2, 3} (3 个)
- 高度 ≥ 1 : {1, 2, 3, 4, 5, 6, 7} (7 个)
- 高度 ≥ 0 : {1, 2, ..., 15} (15 个)

4. 计算“高度恰好为 h ”的结点数

- 高度恰好为 3: $1 - 0 = 1$ (只有结点 1)
- 高度恰好为 2: $3 - 1 = 2$ (结点 2, 3)
- 高度恰好为 1: $7 - 3 = 4$ (结点 4, 5, 6, 7)
- 高度恰好为 0: $15 - 7 = 8$ (结点 8, 9, 10, 11, 12, 13, 14, 15, 即叶子结点)

减去的原因:

高度 $\geq h$ 的结点包含了所有更高的结点, 要得到“恰好为 h ”的结点, 必须把那些高度 $\geq h+1$ 的结点去掉。这样剩下的才是高度正好等于 h 的结点。

复杂度分析

- 含 n 个元素的堆的高度为 $\lceil \lg n \rceil$
- 高度为 h 的结点有 $N_h \leq \left\lceil \frac{n}{2^{h+1}} \right\rceil$
- 把一个无序数组构造一个最大堆：线性

$$T(n) = \sum_{h=0}^{\lceil \lg n \rceil} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O(n \sum_{h=0}^{\lceil \lg n \rceil} \frac{h}{2^h}) = O(n)$$

令： 对一个高度为 h 的结点执行 MAX-HEAPIFY，最坏情况下元素最多向下走 h 层。

$$x = \frac{1}{2}$$

那么：

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \sum_{h=0}^{\infty} h \left(\frac{1}{2}\right)^h = 2$$

公式： $\sum_{h=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$

堆排序 **HEAPSORT**

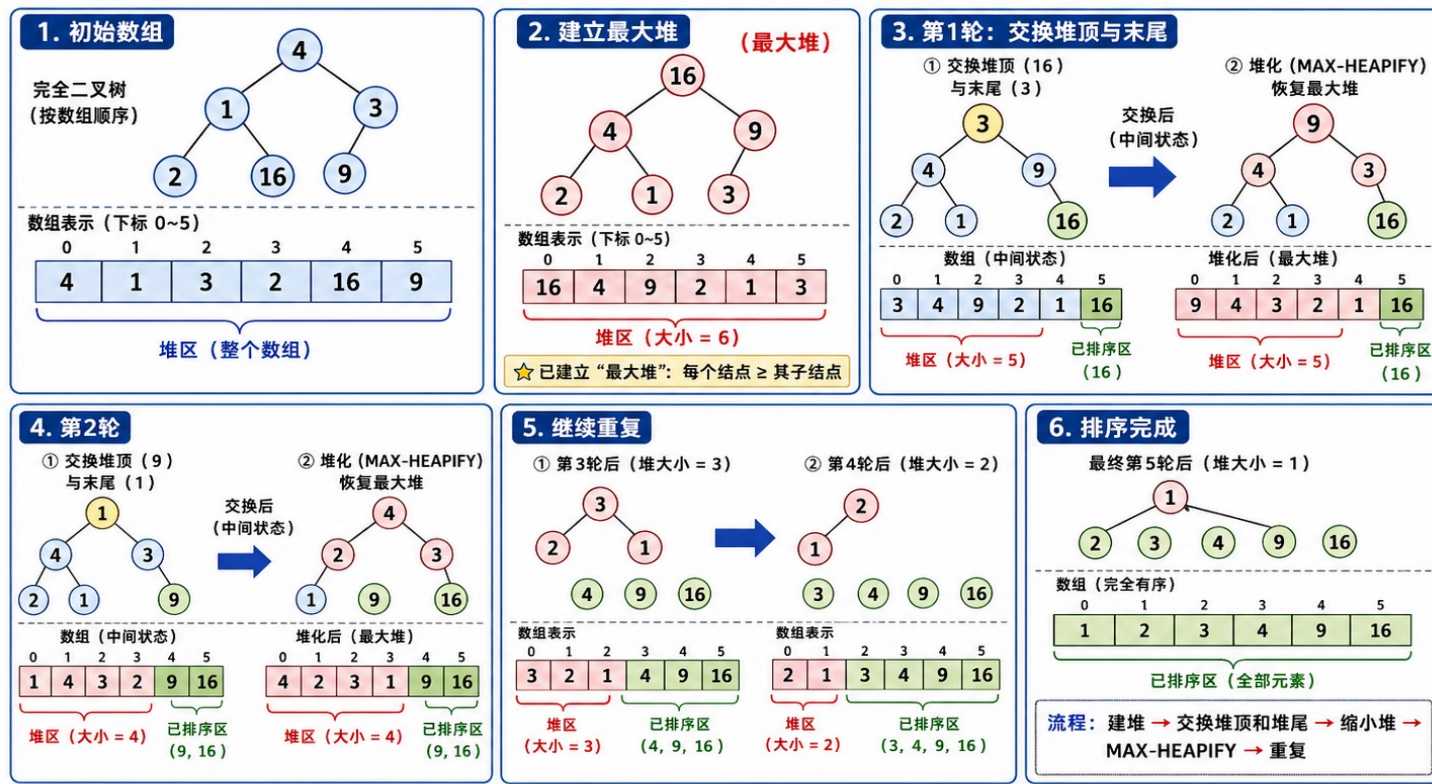
- **排序策略：**

1. 从无序数组中建立最大堆。
2. 找到最大元素 $A[1]$ 。
3. 交换元素 $A[n]$ 和 $A[1]$ ：现在最大元素位于数组的末尾
4. 从堆中删除末尾节点 n （通过减少堆的大小变量）。
5. 新的根节点可能违反最大堆的性质，但它的子节点仍是最大堆。运行 MAX-HEAPIFY来修复问题
6. 如果堆不为空，则回到步骤 2。

堆排序算法

堆排序 (HEAPSORT)

核心思想：先建最大堆，再反复把堆顶最大值放到数组末尾



- 经过 n 次迭代后，堆将变为空。
- 每次迭代都涉及一次交换操作和 MAX-HEAPIFY 操作；因此，每次迭代的时间复杂度为 $O(\log n)$
- 总体运行时间： $O(n \log n)$
- 稳定排序：相同元素，排序前后相对顺序不变；不稳定排序：可能改变。
- 堆排序是不稳定排序
- 归并排序是稳定排序
 - 合并时需要定义规则

Top-k 算法

- 用堆找出Top-k ?

Top-k 算法

1. 前 k 个元素建立最小堆
 1. 复杂度为 $O(k)$
2. 对剩下的 $n - k$ 个元素逐个扫描：
 1. 与堆顶比较： $O(1)$
 2. 如果不大于堆顶：直接跳过
 3. 如果大于堆顶：替换堆顶，并重新调整堆： $O(\log k)$

- 因此最坏情况下：

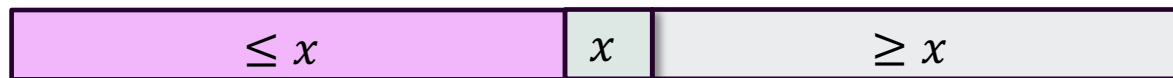
$$T(n, k) = O(k) + (n - k) \cdot O(\log k)$$

- 所以： $O(n \log k)$

快速排序

```
QUICK-SORT (A, l, r) //数组A[l,...,r]
    if l < r then
        x ← Partition(A, l, r)
        QUICK-SORT(A, l, x-1)
        QUICK-SORT(A, x+1, r)
```

1. 分解：将数组以**区分点**进行划分，比**区分点**小的点都在左数组，比**区分点**大的点都在右数组
2. 解决：递归地排序两个子序列
3. 合并：无需



区分点

Partition 子算法

```
Partition (A, p, q)  //数组A[p,...,q]
    x ← A[p]
    i ← p
    for j ← p+1 to q
        if A[j] ≤ x then
            i ← i+1
            exchange A[i] and A[j]
    exchange A[i] and A[p]
    return i
```

输入数组长度为 n 的算法复杂度为 $\Theta(n)$

中间状态：



最坏情况下时间复杂度

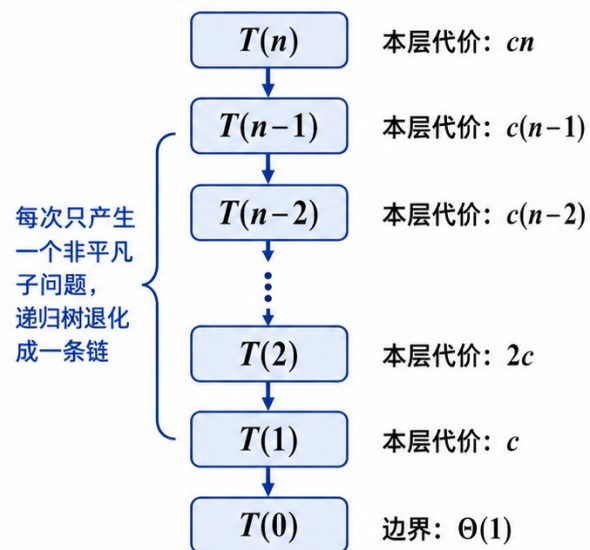
- 输入数据已经排序或逆序的情形
- 每次都选取数组中的最小或最大元素作为分区点
- 一侧的分区数组为空

$$T(n) = T(n - 1) + \Theta(n)$$

求解递归树

递归树求解： $T(n) = T(0) + T(n-1) + \Theta(n)$

因为 $T(0) = \Theta(1)$ ，可视为 $T(n) = T(n-1) + cn$ ，
因此递归树退化成一个链（线性链）。



$$T(n) = cn + c(n-1) + c(n-2) + \cdots + 2c + c + \Theta(1)$$

$$= c(1 + 2 + \cdots + n) + \Theta(1)$$

$$= c \cdot \frac{n(n+1)}{2} + \Theta(1)$$

$$= \Theta(n^2)$$

结论： $T(n) = \Theta(n^2)$

平均情况下的时间复杂度

- 最开始输入情况：有 n 种输入可能

x 排好的位置	子问题1规模	子问题2规模
1	0	$n-1$
2	1	$n-2$
3	2	$n-3$
...	...	...
$n-1$	$n-2$	1
n	$n-1$	0

对每个输入，划分的比较次数都为 $n-1$

平均情况下的时间复杂度

- n 种输入是等概率的

$$T(0) + T(n-1) + n - 1$$

$$T(1) + T(n-2) + n - 1$$

$$T(2) + T(n-3) + n - 1$$

...

$$T(n-2) + T(1) + n - 1$$

$$+ T(n-1) + T(0) + n - 1$$

$$2[T(1) + T(2) + \cdots + T(n-1)] + n(n-1)$$

递归式如下

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + \Theta(n)$$

$$T(1) = 0$$

求解技巧

- 利用两个方程相减，将右边的项尽可能消去，以达到降阶的目的

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

求解递归式

$$\begin{aligned} nT(n) - (n-1)T(n-1) \\ = 2T(n-1) + cn^2 - c(n-1)^2 \end{aligned}$$

化简

$$nT(n) = (n+1)T(n-1) + \underbrace{2cn - c}_{\Theta(n)}$$

两边除以
 $n(n+1)$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c'}{n+1}$$

$$\begin{aligned} \frac{T(n)}{n+1} &= \frac{T(n-1)}{n} + \frac{c'}{n+1} = \dots \\ &= c' \left(\frac{1}{n+1} + \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{3} \right) + \frac{T(1)}{2} \\ &= c' \left(\frac{1}{n+1} + \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{3} \right) \\ &= \Theta(\ln n) \end{aligned}$$

$$T(n) = O(n \log n)$$

快排为什么可以递归?

关键不是“变小”，而是子问题与原问题具有相同结构。



两个子问题，仍然都是“排序问题”

分治思想



分治法： 将一个大问题拆成若干个规模更小、形式相同的子问题，递归求解，再组合为原问题的解。

分治算法的一般模板

```
DivideAndConquer(P):
```

```
  if P 足够小:
```

```
    return 直接求解(P)
```

```
  将 P 分成  $P_1, P_2, \dots, P_a$ 
```

```
  for each  $P_i$ :
```

```
     $S_i = \text{DivideAndConquer}(P_i)$ 
```

```
  return  $\text{Combine}(S_1, S_2, \dots, S_a)$ 
```

识别分治的三个检查点

- 1 子问题是否比原问题更小?
- 2 子问题是否保持同样的结构?
- 3 子问题的解如何组合?

边界条件决定递归何时停止

递归是一种实现方式，分治是一种设计思想

递归，但通常不称为分治

factorial(n)

$$n! = n \cdot (n - 1)!$$

只产生一个规模 $n-1$ 的子问题

典型的分治递归

QuickSort(A)

$$T(n) = T(k) + T(n - k - 1) + \Theta(n)$$

产生多个结构相同的子问题

因此：使用递归 $\neq$ 一定使用了分治

同样是分治，工作量可以落在不同阶段

阶段	快速排序	归并排序
Divide	难：需要 Partition	易：从中间切开
Conquer	递归排序左右部分	递归排序左右部分
Combine	易：几乎无需额外工作	难：需要 Merge

快速排序：难分，易合

归并排序：易分，难合

把问题拆小，就一定会更快吗？

不一定。

效率取决于拆分后的形状与每层付出的代价。

01 拆成多少个子问题？

02 每个子问题有多大？

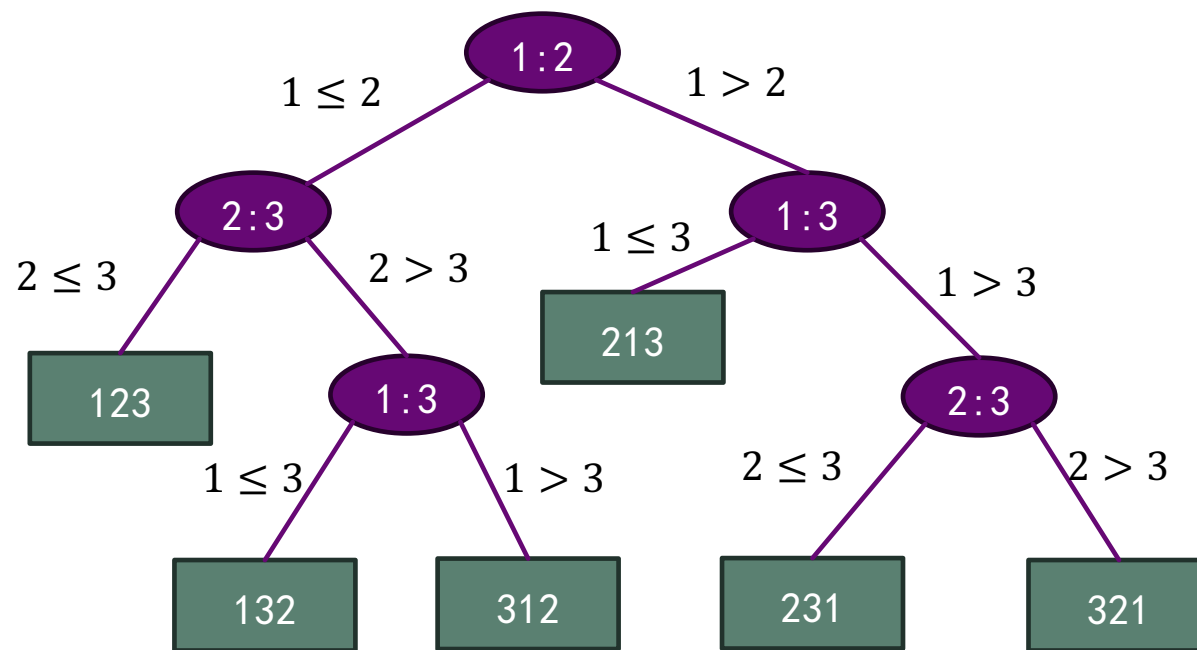
03 分与合本身有多贵？

比较排序算法的极限

- 目前我们学习的所有算法都是比较排序算法
 - 插入排序
 - 归并排序
 - 快速排序
 - 堆排序
- 最坏情况下的时间复杂度（最好）： $O(n \log n)$
- $O(n \log n)$ 是不是最优的比较排序算法复杂度？
- 换句话说，能否分析某类算法的极限在哪？

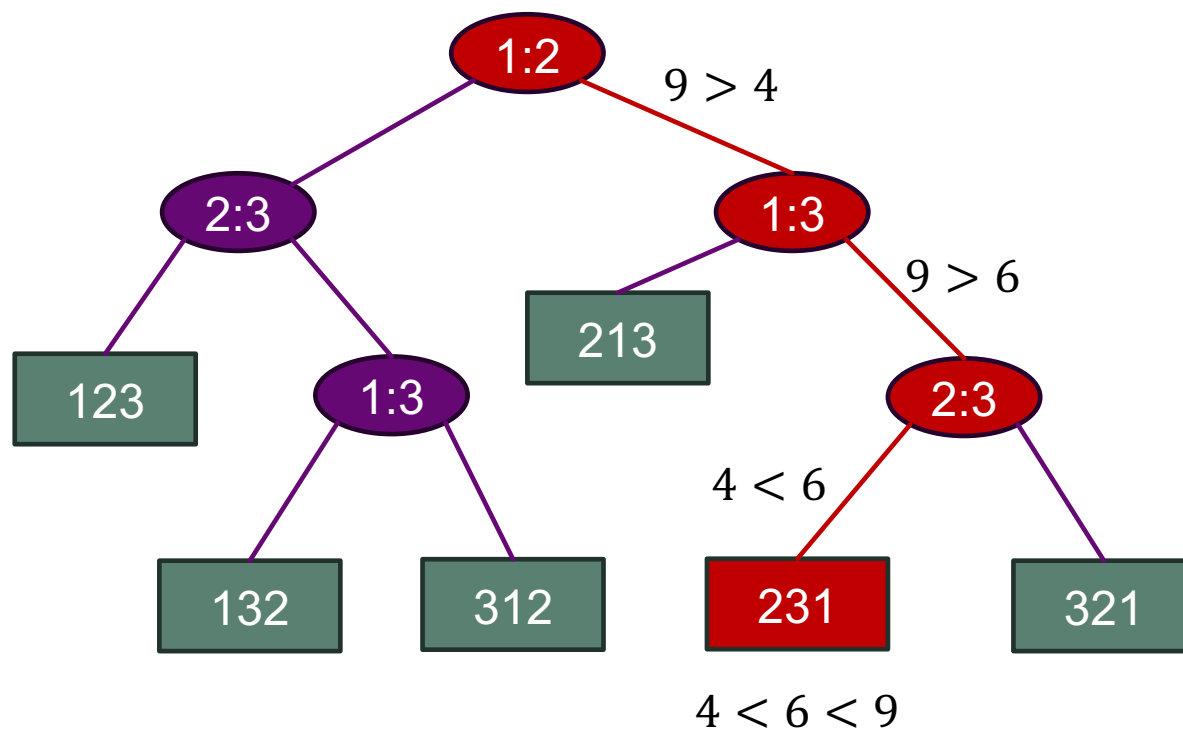
决策树模型

- 排序 $\langle a_1, \dots, a_n \rangle$
- 每个内部结点表示为 $i:j$
 - 左子树表示当 $a_i \leq a_j$ 时接下来的比较
 - 右子树表示当 $a_i > a_j$ 时接下来的比较



决策树模型的实例

- 排序 $\langle a_1, a_2, a_3 \rangle = \{9, 4, 6\}$
- 每一个叶子结点是一个排列
- $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$,
- 满足 $a_{\pi(1)} \leq a_{\pi(2)} \leq a_{\pi(3)} \leq \dots \leq a_{\pi(n)}$



决策树模型

- 决策树可以**模拟任何比较排序算法**的执行过程：
 - 每个输入大小 n 对应一棵树。
 - 将算法视为每次比较两个元素时进行的分裂。
 - 这棵树包含了所有可能指令路径上的比较。
 - **算法的运行时间 = 所走路径的长度。**
 - 最坏情况下的运行时间 = **树的高度**。

比较排序算法的下界

- **定理** 在最坏情况下，任何比较算法都需要做 $\Omega(n \lg n)$ 次比较
- **证明：** 对于 n 个互不相同的元素，共有 $n!$ 种可能的排列。比较排序的决策树必须能够区分所有这些排列，因此决策树至少需要 $n!$ 个叶子结点，即 $L \geq n!$ 。另一方面，高度为 h 的二叉决策树最多有 2^h 个叶子，因此 $n! \leq 2^h$ 。

$$h \geq \lg(n!) \geq \lg\left((n/e)^n\right) = n \lg n - n \lg e = \Omega(n \lg n)$$

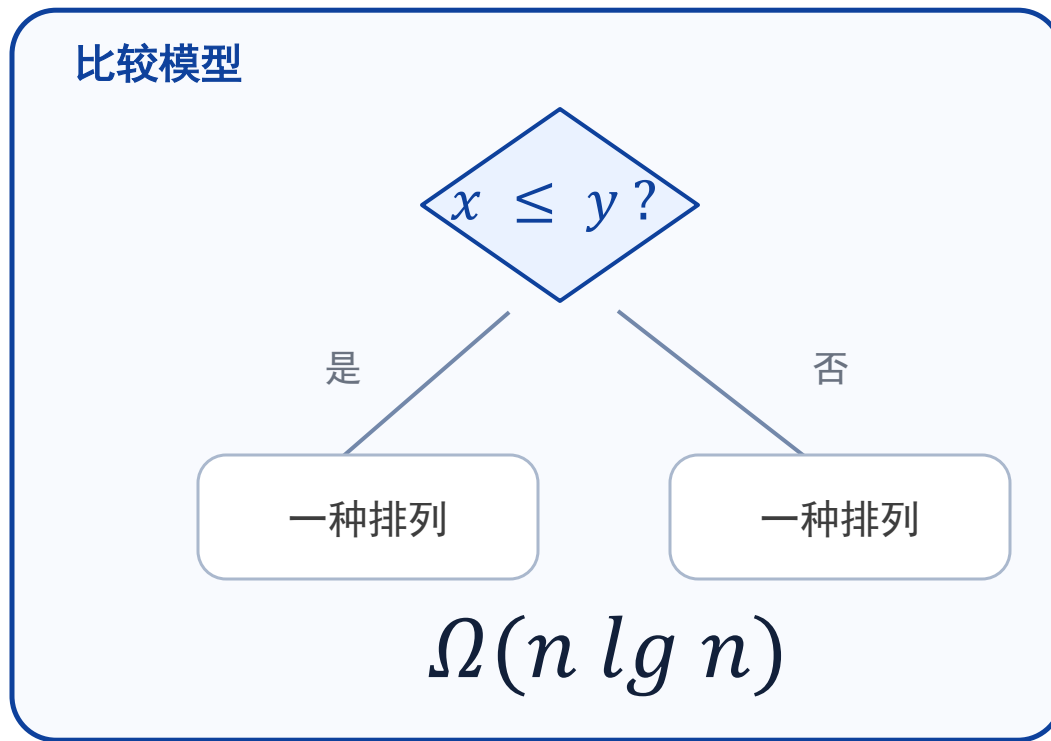
Stirling 公式：对阶乘 $n!$ 的一个非常经典的近似

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

比较排序下界的适用范围

下界成立需要三个前提

- 仅通过元素比较获得顺序信息
- 需要处理任意互不相同的元素
- 考察最坏情况下的比较次数



比较排序下界只约束比较模型

非比较排序为何能突破下界

计数排序直接利用键值范围，不依赖两两比较。



适用条件 $0 \leq A[i] \leq k$ 键值范围已知且不大

时间与空间 $T(n, k) = O(n + k)$ $S(k) = O(k)$

突破下界，依靠增加输入假设与其他可用操作

为什么讲下界？

- 算法设计：
“我能做到多快？”
- 复杂度分析：
“这个算法到底有多快？”
- 算法下界：
“任何算法最多能有多快？”

上界给出可能，下界揭示极限

现代算法的思想：大模型推理优化为例

- 为什么 ChatGPT 不能一次把整段答案直接生成出来？



Autoregressive decoding is sequential.

每个新 token 都依赖已经生成的前缀

生成 100 个 token，是否一定需要约 100 次昂贵的大模型解码？

把问题抽象成一个算法问题

输出依赖链

1

$$F(x_1) \rightarrow x_2$$

2

$$F(x_1, x_2) \rightarrow x_3$$

3

$$F(x_1, x_2, x_3) \rightarrow x_4$$

真正的瓶颈



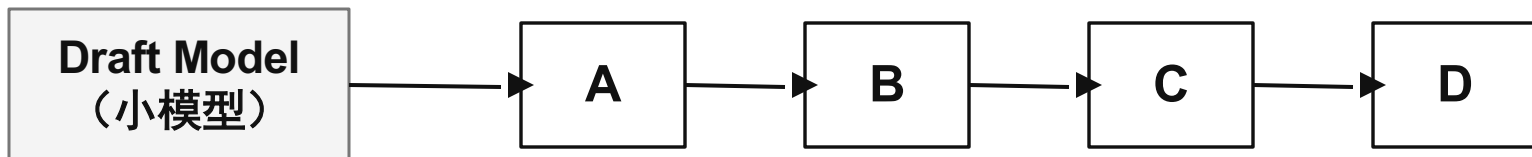
昂贵操作

大量昂贵操作
必须串行执行

如果单次昂贵计算已经很难再加速，能否让它少调用几次？

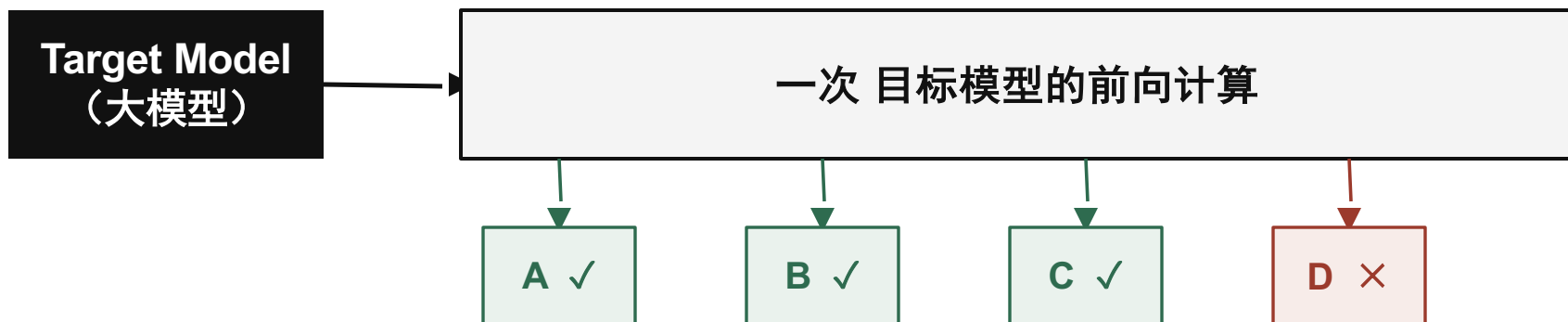
核心思想：先猜，再验证

便宜地猜



小模型连续猜出多个候选

昂贵地验证



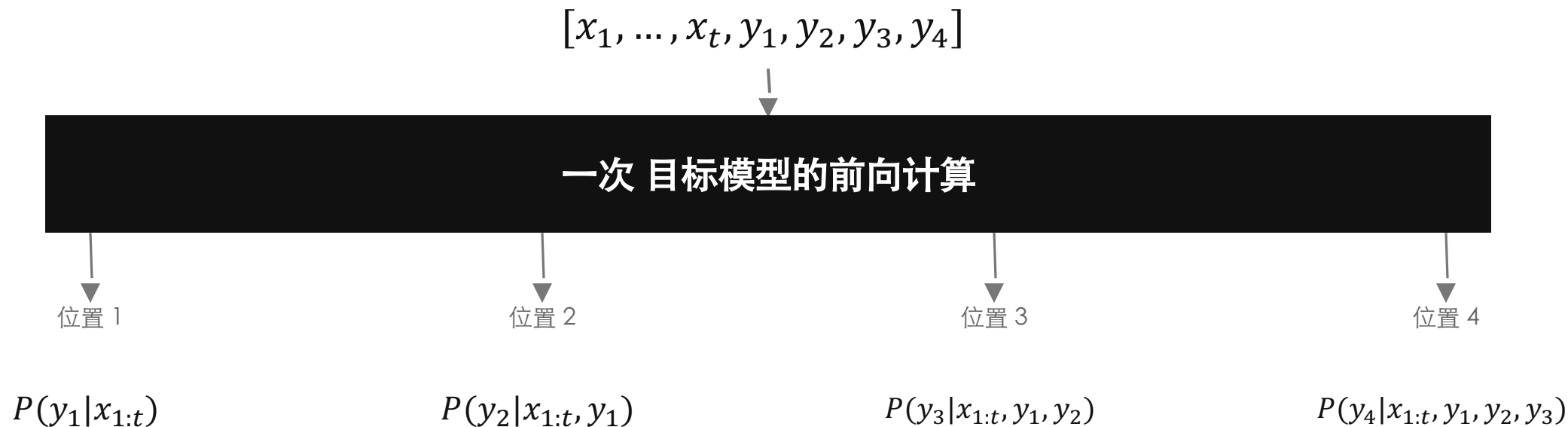
1 次

Target forward
推进 3 个 token

用便宜计算，减少昂贵计算的调用次数。

一次 前向计算 如何验证多个位置?

候选 token 已经知道，可以把完整序列一次输入 目标模型



通过Casual Mask，模型计算中（Self-Attention）概率可以批量进行

批量前向计算

Causal Mask 的作用：每个位置只能使用它左边已经出现的 token。

每个候选位置能看到什么？

检查 y_1	可看	$x_1 \dots x_t$
检查 y_2	可看	$x_1 \dots x_t, y_1$
检查 y_3	可看	$x_1 \dots x_t, y_1, y_2$
检查 y_4	可看	$x_1 \dots x_t, y_1, y_2, y_3$

候选区的 Causal Mask

	y_1	y_2	y_3	y_4
y_1	✓	✗	✗	✗
y_2	✓	✓	✗	✗
y_3	✓	✓	✓	✗
y_4	✓	✓	✓	✓

4 个位置可以一起算；但每个位置只能看自己的左侧前缀。

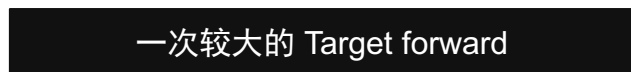
一起计算 $\neq$ 互相看未来；Mask 让每一行只使用合法前缀。

普通逐 token 生成



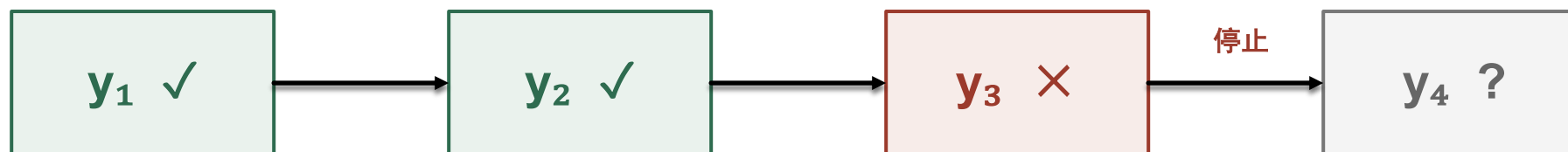
4 次串行 forward

Speculative 验证



批量算概率；accept / reject 仍从左到右

接受与拒绝仍然需要从左到右



y_4 使用的条件前缀

$x_{1:t}, y_1, y_2, y_3$

y_3 被拒绝后, y_4 的条件前缀已经失效, 因此不能直接采用。

昂贵部分

目标模型的前向计算 一次性完成全部候选概率

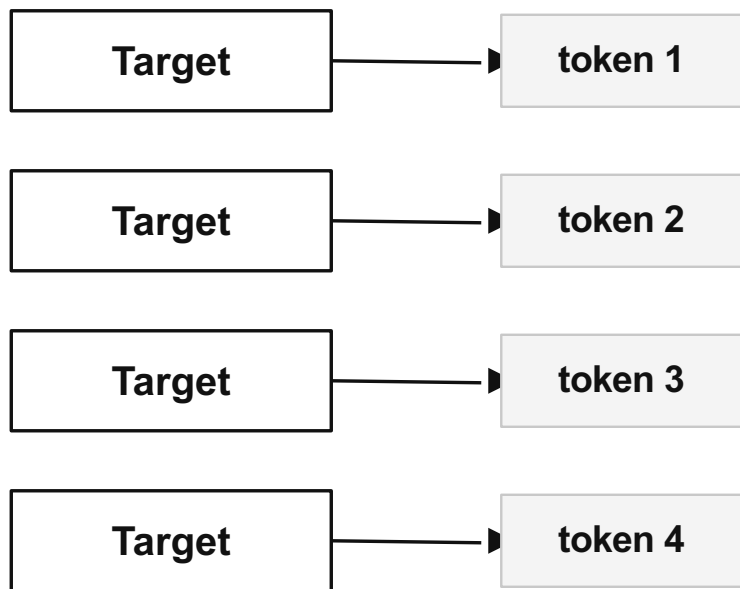
便宜部分



accept / reject 仍按顺序判断

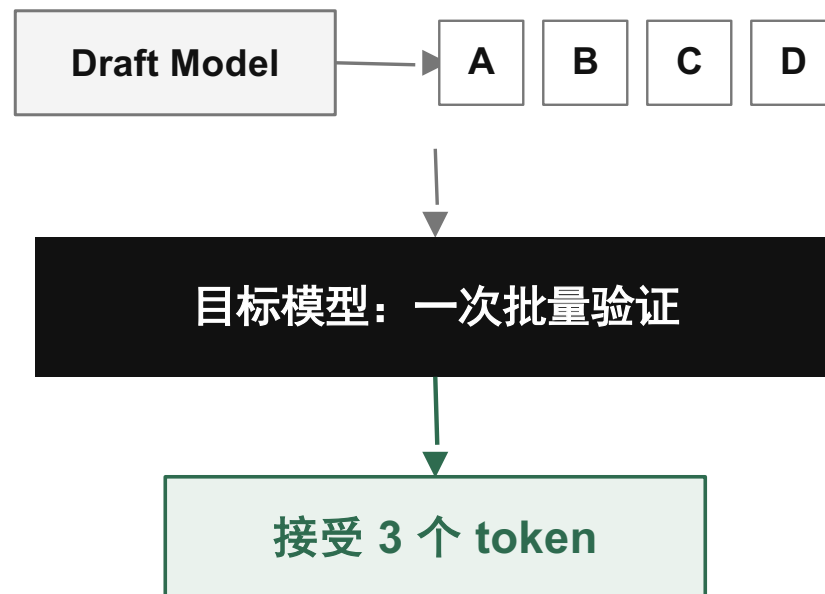
Speculative Decoding 到底快在哪里？

传统 Autoregressive Decoding



4 次昂贵 前向计算

Speculative Decoding



1 次昂贵 前向计算

它主要降低端到端耗时和昂贵串行调用次数，生成成本仍随输出长度增长

从 **Speculative Decoding** 学到的算法思想

01

Identify the Bottleneck

找到真正昂贵的操作

02

Cheap Speculation

用便宜方法提前预测

03

Batch Verification

把多个验证合并到一次昂贵计算

04

Preserve Correctness

加速过程仍需满足原算法的正确性要求

参考文献: Leviathan Y, Kalman M, Matias Y. Fast inference from transformers via speculative decoding[C]//International conference on machine learning. PMLR, 2023: 19274-19286.

算法课不是背“标准答案”

- 今天我们讲了排序、递归、复杂度，也看了 Speculative Decoding。
 - 这些例子真正想训练的，不是让大家记住多少算法名字。更重要的是，面对一个新问题时，你能不能问出几个问题：最贵的操作是什么？有没有重复计算？能不能提前猜？能不能并行？能不能用空间换时间？优化之后正确性还在不在？
 - 算法课真正训练的，是这种思考方式。
-
- 不要只记住算法，试着记住它为什么会被这样设计。

练习题

■ 下面哪些说法一定正确？已知 $f(n) = \Theta(n \log n)$ ，判断以下结论，并说明理由：

- A. $f(n) = O(n^2)$
- B. $f(n) = O(n)$
- C. $f(n) = \Omega(n)$
- D. $f(n) = \Omega(n^2)$
- E. $f(n) = o(n^2)$
- F. $f(n) = \omega(n)$

命题	对错
A	✓
B	✗
C	✓
D	✗
E	✓
F	✓

练习题

■ 解递归式

$$T(n) = T(n/2) + T(n/4) + n, T(1) = \Theta(1).$$

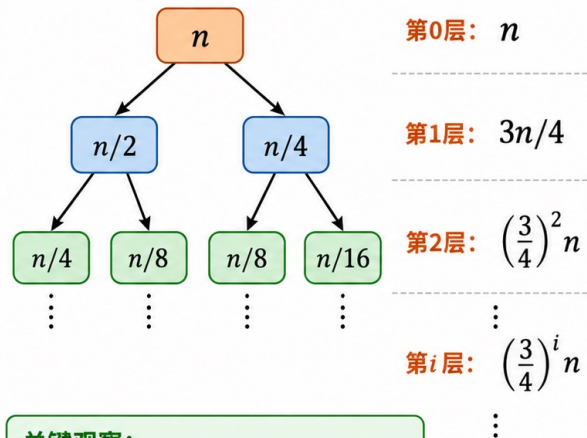
■ 要求：

- 用递归树分析 $T(n)$;
- 给出尽可能紧的渐近界。

递归树分析: $T(n) = T(n/2) + T(n/4) + n$

基例: $T(1) = \Theta(1)$

递归树



关键观察:

因为每个规模 x 会分成 $x/2$ 和 $x/4$, 所以下一层总代价变为 $3x/4$ 。

1 看每层总代价

每一层的总代价都按比例 $3/4$ 缩小, 因此形成几何级数。

2 求非叶子层总和

非叶子层总代价 = $n \sum_{i=0}^{h-1} \left(\frac{3}{4}\right)^i \leq 4n = O(n)$ 。

3 看树高

最长路径是不断取 $n/2$, 因此树高 $h = O(\lg n)$ 。

4 看叶子层

每个叶子代价为 $\Theta(1)$ 。若记叶子数为 $L(n)$, 则

$$L(n) = L(n/2) + L(n/4), \quad L(1) = 1,$$

因此 $L(n) = O(n^{0.694\dots}) \subseteq O(n)$ 。所以叶子层总代价也是 $O(n)$ 。

5 合并上下界

上界: $O(n)$; 下界: 根结点代价就是 n , 因此 $T(n) = \Omega(n)$ 。

结论: $T(n) = \Theta(n)$

练习题

■ 解递归式

$$T(n) = T(\sqrt{n}) + 1, T(2) = \Theta(1).$$

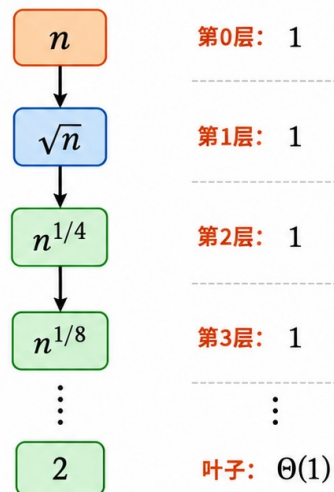
■ 要求：

- 用递归树分析 $T(n)$;
- 给出尽可能紧的渐近界。

递归树分析： $T(n) = T(\sqrt{n}) + 1$

基例： $T(2) = \Theta(1)$

递归树



关键观察：

每次递归，问题规模从 x 变为 $\sqrt{x}$ ，
所以递归树不是分叉，而是一条链。
每一层的非递归代价都是 1。

1 看每层代价

每一层只有一个子问题，且该层的非递归代价都是 1。

第 i 层代价 = 1

2 看第 i 层的规模

连续开平方 i 次后，规模变为：

$$n^{1/2^i}$$

3 看树高

递归在规模降到 2 时停止，因此令：

$$n^{1/2^h} = 2$$

$$\text{取 lg: } (\lg n)/2^h = 1$$

$$\text{所以 } 2^h = \lg n$$

$$\text{因此 } h = \lg \lg n$$

4 求总代价

非叶子层共有 $\Theta(\lg \lg n)$ 层，每层代价为 1；叶子代价为 $\Theta(1)$ 。

$$\begin{aligned} T(n) &= 1 + 1 + \cdots + 1 + \Theta(1) \\ &= \Theta(\lg \lg n) \end{aligned}$$

5 合并上下界

上界和下界一致，因此最终得到精确渐近界。

$$\text{上界: } O(\lg \lg n)$$

$$\text{下界: } \Omega(\lg \lg n)$$

结论： $T(n) = \Theta(\lg \lg n)$

大作业

- 发布在 elearning 上，通过 elearning 提交（有deadline）